

Delivery Cloud

Plataforma de delivery basada en microservicios con analitica sobre un data lake, desplegada en AWS

Informe tecnico — Proyecto Parcial (Semanas 3 a 6)

Elemento	Enlace
Repositorio publico (GitHub)	https://github.com/miguelespinozaa-ship-it/ProeyctParcialProp
Aplicacion web (AWS Amplify)	https://main.d2wzxjgeu63fi4.amplifyapp.com
API publica (API Gateway, HTTPS)	https://jt3z0elcud.execute-api.us-east-1.amazonaws.com
Fecha	21 de septiembre de 2026

1. Resumen

Delivery Cloud es una plataforma tipo Uber Eats que armamos como proyecto del curso. Maneja tres tipos de usuario: los clientes que exploran restaurantes y hacen pedidos, los admins de restaurante que gestionan su menu y despachan pedidos, y los repartidores que toman entregas disponibles y las cierran.

El backend son cinco microservicios corriendo en Docker, escritos en tres lenguajes distintos (Python, Java y Node.js) y con tres motores de base de datos diferentes (MySQL, PostgreSQL y MongoDB). Aparte del flujo transaccional, montamos un pipeline de Data Science: cada tabla se replica al 100% en un data lake en S3, AWS Glue las cataloga automaticamente, y desde Amazon Athena podemos consultar las tres fuentes con SQL. El microservicio analitico (MS5) es el que expone estos resultados al frontend.

Todo el sistema esta desplegado en AWS: dos maquinas virtuales de aplicacion balanceadas con un ALB interno, una maquina de bases de datos privada, una maquina dedicada a la ingesta, un API Gateway con HTTPS como unico punto publico, y el frontend en AWS Amplify con CI/CD conectado a GitHub.

Microservicios	Lenguajes / motores	Registros cargados	Maquinas virtuales	Consultas Athena
5 en Docker	Python, Java, Node.js — MySQL, PostgreSQL, MongoDB	Mas de 20,000 por base de datos	4 EC2 (2 de App balanceadas)	4 con JOIN y 2 vistas

Cumplimiento de los requisitos

Un resumen rapido de como cumplimos cada punto del enunciado y donde lo puede verificar el profesor:

Requisito	Como se cumple	Seccion
5 microservicios en Docker; 3 con su propia BD, 3 lenguajes, 3 BD (2 SQL y 1 NoSQL)	MS1 FastAPI + MySQL, MS2 Spring Boot + MongoDB, MS3 Express + PostgreSQL	3
Diagrama E/R por BD SQL y JSON de la BD NoSQL; minimo 2 tablas relacionadas	E/R de MySQL (usuarios 1:N direcciones) y PostgreSQL (orders 1:N order_items); estructura JSON de restaurantes	4
Un microservicio consume a otro	MS3 consulta a MS2 (validar plato y precio) y a MS1 (nombres de clientes)	3.2
Un microservicio sin BD que solo consume a otros	MS4 Agregador/Rastreo: tracking y 3 dashboards con datos de MS1, MS2 y MS3	3
Un microservicio analitico con queries en Athena	MS5 ejecuta consultas con boto3 sobre las vistas del data lake	7
Carga masiva unica, minimo 20,000 registros por BD	seed.py (usuarios), seed.js (pedidos) y seed.js en mongosh (restaurantes)	4.4
2 VMs de produccion con balanceador privado, BD en una tercera VM privada, APIs publicas con HTTPS en API	2 EC2 de App detras de un ALB interno; API Gateway apunta al ALB via VPC Link; la BD no expone puertos a internet	6

Requisito	Como se cumple	Seccion
Gateway		
Documentar las 5 APIs en Swagger UI	Swagger UI accesible en los 5 servicios, incluso a traves del API Gateway	6.5
Aplicacion web que consuma los 5 servicios (mas de 2 metodos REST por servicio) desplegada en Amplify	SPA React que hace 5, 9, 8, 4 y 2 llamadas REST a MS1 hasta MS5; desplegada en AWS Amplify con CI/CD desde GitHub	5
MV de ingesta, bucket S3 y 3 contenedores Python de ingesta (pull del 100%)	MV PP-Ingest-VM con 3 contenedores; el S3 recibe snapshot completo por tabla	7.1
Catalogo en Glue y diagrama E/R que relacione las tablas del catalogo	Crawler sobre S3 crea 5 tablas; diagrama E/R del catalogo incluido	7.2
4 consultas SQL con JOIN en Athena y 2 vistas	Ejecutadas en Athena con evidencia (ID de ejecucion y resultados)	7.3
Diagrama de arquitectura en draw.io con todos los servicios AWS	infra/diagramas/proyecto-delivery-cloud.drawio (5 paginas)	2
Repositorios publicos en GitHub	Un monorepo publico con todo el codigo fuente	Anexo A

2. Arquitectura de la solucion

La solucion completa vive en la region us-east-1 de AWS. En el diagrama se ven las tres capas: el frontend en Amplify (que es publico via HTTPS), el backend detras de un ALB interno (que no tiene IP publica), y el pipeline de Data Science que corre en paralelo alimentando el data lake.

Diagrama de Arquitectura de Solución — Delivery Cloud (AWS · us-east-1)

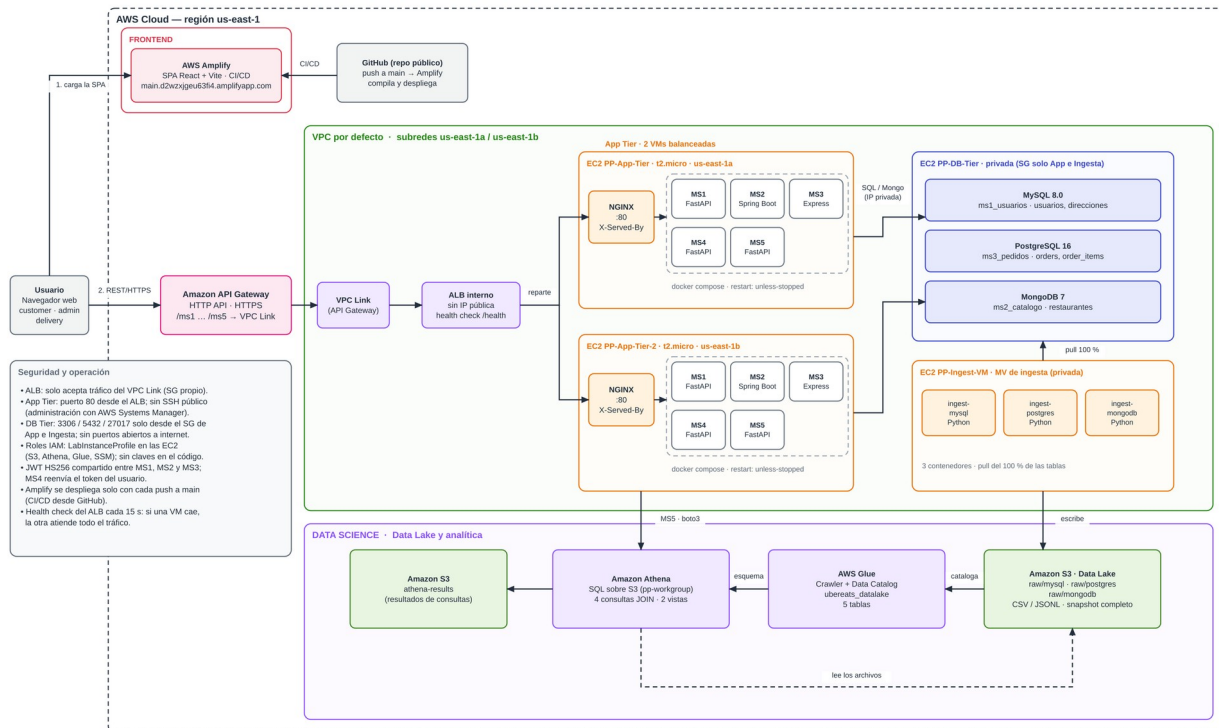


Figura 1. Arquitectura de solucion en AWS (editable en infra/diagramas/proyecto-delivery-cloud.drawio)

Flujo de una peticion

Asi viaja una peticion desde el cliente hasta la base de datos:

- 1. Carga de la aplicacion.** El navegador descarga la SPA React desde AWS Amplify por HTTPS.
- 2. Llamada a la API.** La SPA llama al API Gateway (HTTP API, HTTPS) con las rutas /ms1 hasta /ms5 y el token JWT del usuario.
- 3. Entrada a la red privada.** El API Gateway entra al VPC por un VPC Link que apunta al ALB interno (sin IP publica; solo acepta trafico del VPC Link).
- 4. Balanceo.** El ALB reparte la peticion entre las dos VMs de App (zonas us-east-1a y 1b) y saca de rotacion la que falle su health check.
- 5. Enrutamiento.** El NGINX de cada VM enruta por prefijo de ruta al microservicio correspondiente y agrega la cabecera X-Served-By con el nombre de la VM.
- 6. Datos.** Cada microservicio se conecta por IP privada a su base de datos en la VM de BD; MS5 consulta a Athena con boto3 usando el rol IAM de la instancia.

Decisiones de diseno

- Base de datos por servicio.** Cada micro es dueño de su schema. Las relaciones entre servicios (customer_id, restaurant_id) son referencias logicas y se resuelven por API, no por FK cruzadas.
- JWT compartido.** MS1 emite tokens HS256 y MS2/MS3 los validan con el mismo secreto. MS4 reenvia el token al llamar a los otros micros.

- **Servicios sin estado.** Toda la persistencia esta en la VM de BD. Por eso las dos VMs de App son identicas y pueden balancearse sin coordinacion.
- **Paginado opcional.** Los listados grandes aceptan ?page= y ?page_size=; sin esos parametros devuelven todo el conjunto (como antes) para no romper consumidores existentes.

3. Microservicios

Servicio	Tecnologia	Base de datos	Responsabilidad	Operaciones (Swagger)
MS1 Usuarios/Auth	Python + FastAPI	MySQL 8.0 (usuarios, direcciones)	Registro, login (JWT), perfil, direcciones y consulta de usuarios	12
MS2 Catalogo	Java 17 + Spring Boot	MongoDB 7 (restaurantes con platos y resenas embebidos)	Restaurantes, menu (CRUD del admin) y resenas	13
MS3 Pedidos	Node.js + Express	PostgreSQL 16 (orders, order_items)	Crear pedido, cambiar de estado, tomar y entregar, resumen por estado	9
MS4 Agregador/Rastreo	Python + FastAPI	Sin base de datos	Tracking de un pedido y dashboards por rol combinando MS1, MS2 y MS3	5
MS5 Analitico	Python + FastAPI + boto3	Amazon Athena (data lake)	Top de restaurantes, metricas de usuarios y resumen de ventas	5

3.1 Estados y reglas de negocio

- **Pedido:** PEDIDO → ENVIADO (lo cambia el admin dueño del restaurante) → ENTREGADO (lo cierra el delivery que lo tomo). Un pedido en ENVIADO sin repartidor esta disponible para que un delivery lo tome (claim); si dos lo intentan a la vez, solo uno lo obtiene.
- **Roles:** El registro publico no permite crear admins. Cada admin esta asociado a un restaurante (via restaurante_id en su token) y solo puede tocar su propio menu y sus pedidos.
- **Resenas:** Puntuacion de 1 a 5 y comentario obligatorio; al guardar una resena se recalcula la calificacion promedio del restaurante.

3.2 Comunicacion entre microservicios

Origen	Destino	Para que
MS3 Pedidos	MS2 Catalogo	Validar que el plato existe y obtener su precio antes de guardar el pedido
MS3 Pedidos	MS1 Usuarios	Resolver nombres de los clientes de un restaurante (en bloques de 200 ids)
MS4 Agregador	MS1, MS2 y MS3	Componer el tracking del pedido y los dashboards de customer, delivery y admin; si un servicio falla responde con datos parciales o un 502 controlado
MS5 Analitico	Amazon Athena	Consultas SQL sobre las vistas

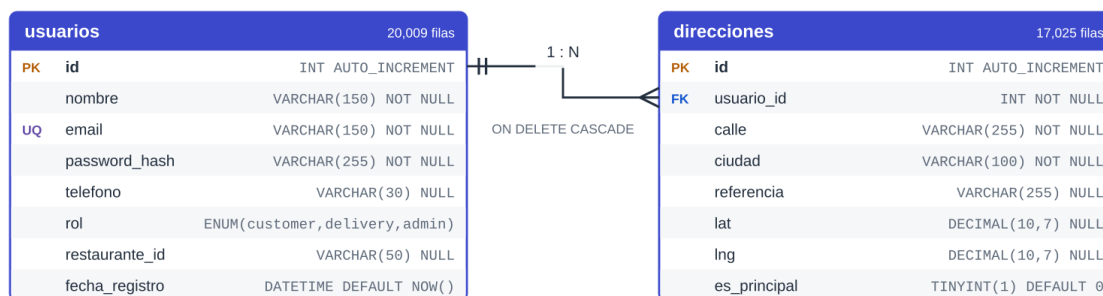
Origen	Destino	Para que
		v_resumen_ventas_restaurante y v_metricas_usuarios

4. Modelo de datos

Cada base de datos SQL tiene dos tablas relacionadas y su diagrama Entidad-Relacion. La base NoSQL se documenta con la estructura JSON de su documento principal.

4.1 MySQL — MS1 (usuarios 1 : N direcciones)

Diagrama Entidad-Relación — MS1 Usuarios/Auth · MySQL 8.0 · base ms1_usuarios



Reglas del modelo

- Un usuario tiene de 0 a N direcciones (usuarios 1 : N direcciones); la FK usuario_id elimina en cascada.
- email es único; rol restringe a customer, delivery o admin. restaurante_id solo lo usa el admin y es una referencia lógica al _id del restaurante en MongoDB (no hay FK entre motores).
- Carga masiva única con seed.py (Faker): 20,000+ usuarios. Índices: idx_usuarios_rol, idx_direcciones_usuario.

Figura 2. Diagrama E/R de MS1.

Reglas del modelo:

- Un usuario tiene de 0 a N direcciones (1:N); la FK usuario_id elimina en cascada.
- El email es unico. El rol restringe a customer, delivery o admin.
- restaurante_id solo lo usa el admin y es una referencia logica al _id del restaurante en MongoDB (no hay FK entre motores).
- Carga masiva con seed.py (Faker): mas de 20,000 usuarios. Indices en (rol) y (usuario_id) de direcciones.

4.2 PostgreSQL — MS3 (orders 1 : N order_items)

Diagrama Entidad-Relación — MS3 Pedidos · PostgreSQL 16 · base ms3_pedidos

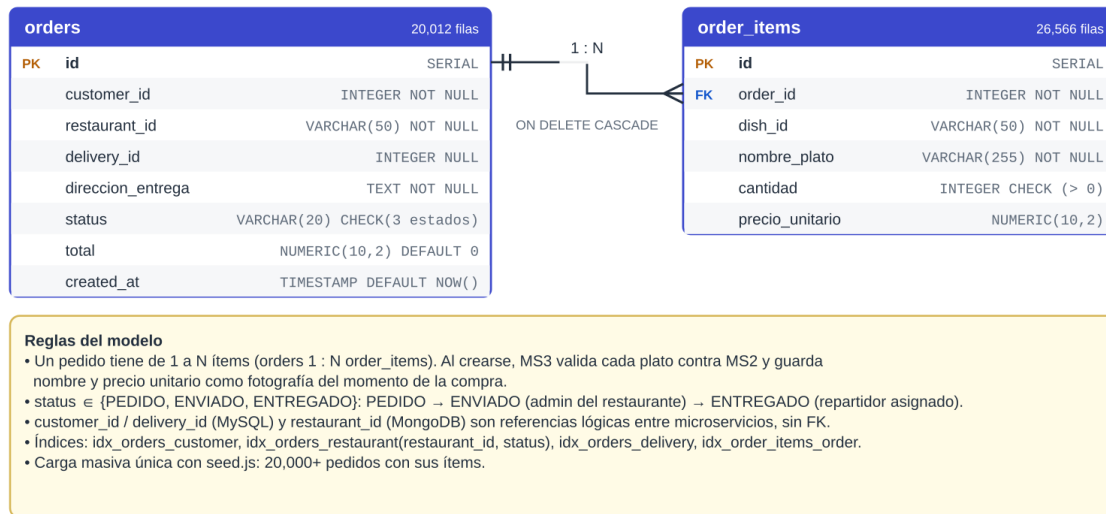


Figura 3. Diagrama E/R de MS3.

Reglas del modelo:

- Un pedido tiene de 1 a N items (orders 1:N order_items). Al crearse, MS3 valida cada plato contra MS2 y guarda nombre y precio unitario como snapshot del momento de la compra.
- status ∈ {PEDIDO, ENVIADO, ENTREGADO}: PEDIDO → ENVIADO (admin del restaurante) → ENTREGADO (repartidor asignado).
- customer_id / delivery_id (MySQL) y restaurant_id (MongoDB) son referencias logicas entre microservicios, sin FK.
- Indices en idx_orders_customer, idx_orders_restaurant(restaurant_id, status), idx_orders_delivery, idx_order_items_order.
- Carga masiva con seed.js: mas de 20,000 pedidos con sus items.

4.3 MongoDB — MS2 (restaurantes con platos y resenas embebidos)

Estructura JSON — MS2 Catálogo de Restaurantes · MongoDB 7 · colección ms2_catalogo.restaurantes

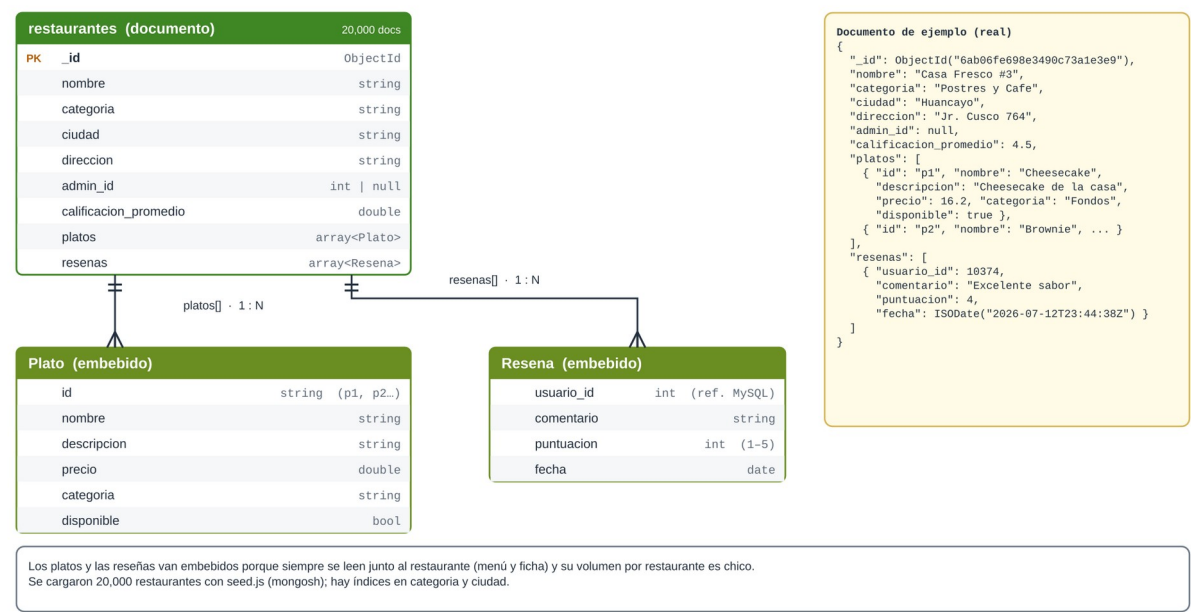


Figura 4. Estructura JSON de la coleccion restaurantes.

Los platos y las reseñas viven embebidos porque siempre se leen junto al restaurante (menu y ficha) y su volumen por restaurante es acotado. Se cargaron 20,000 restaurantes con seed.js (mongosh); hay indices en categoria y ciudad.

4.4 Carga masiva de datos

Los datos ficticios se insertan una sola vez en cada base con scripts incluidos en el repositorio. Los conteos son los del ultimo snapshot cargado al data lake:

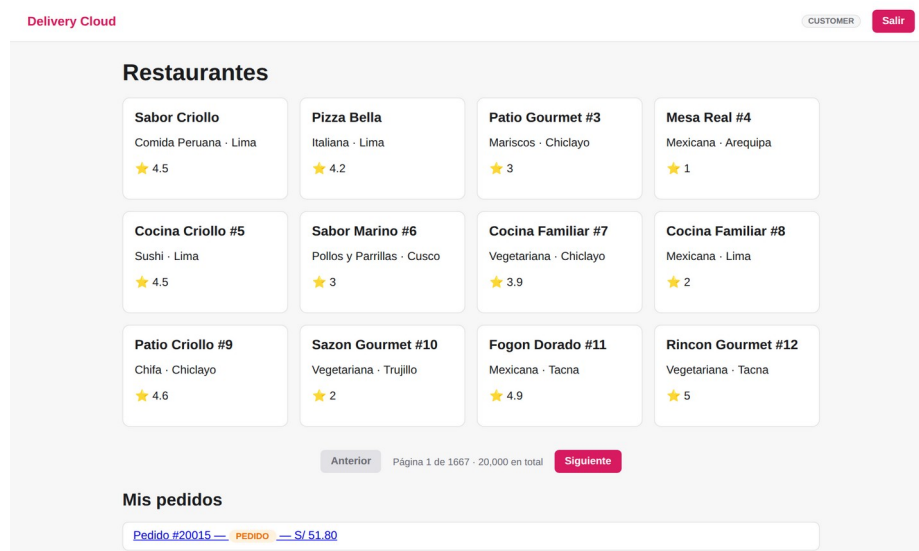
Base de datos	Tabla / coleccion	Script	Registros
MySQL (MS1)	usuarios	seed.py (Faker)	20,007
MySQL (MS1)	direcciones	seed.py	17,025
PostgreSQL (MS3)	orders	seed.js	20,009
PostgreSQL (MS3)	order_items	seed.js	26,566
MongoDB (MS2)	restaurantes	seed.js (mongosh)	20,000

5. Aplicacion web (frontend)

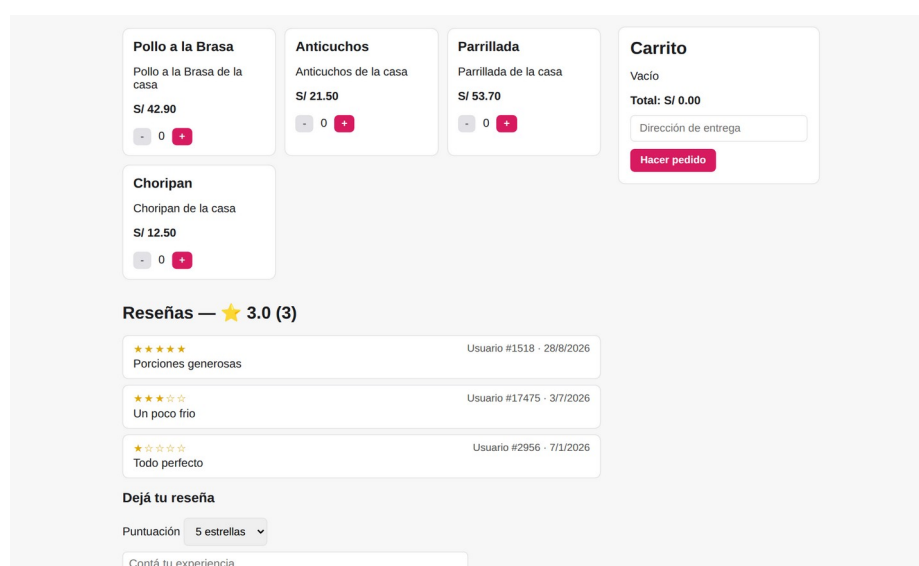
El frontend es una SPA en **React con Vite**, con login y rutas protegidas por rol. Esta desplegada en **AWS Amplify** (<https://main.d2wzxjgeu63fi4.amplifyapp.com>) y consume la API unicamente por el API Gateway con HTTPS.

Rol	Pantallas	Servicios que consume
Customer	Restaurantes (grilla paginada), menu con carrito y resenas, mis pedidos y seguimiento del pedido	MS1, MS2, MS3, MS4
Admin	Panel de pedidos por estado (con platos y cantidades de cada pedido), clientes, gestion del menu y analitica	MS2, MS3, MS4, MS5
Delivery	Pedidos disponibles, entregas en curso, marcar como entregado	MS3, MS4

Pantallas de la aplicacion



Customer: grilla de restaurantes paginada (20,000)



Customer: menu, resenas y carrito

Delivery Cloud

Menú Analítica ADMIN Salir

Sabor Criollo

Av. Siempre Viva 123

PEDIDO (3,336) ENVIADO (3,371) ENTREGADO (3,197)

#19881 — 7 platos — S/ 171.10 — 11525 Gleichner Turnpike

- 4 × Lomo Saltado (S/ 25.90 c/u)
- 3 × Aji de Gallina (S/ 22.50 c/u)

#19880 — 4 platos — S/ 93.40 — 468 Koss Freeway

- 3 × Aji de Gallina (S/ 22.50 c/u)
- 1 × Lomo Saltado (S/ 25.90 c/u)

#19878 — 4 platos — S/ 90.00 — 63869 E State Street

- 4 × Aji de Gallina (S/ 22.50 c/u)

#19875 — 6 platos — S/ 141.80 — 485 Marsh Lane

- 2 × Lomo Saltado (S/ 25.90 c/u)
- 4 × Aji de Gallina (S/ 22.50 c/u)

#19867 — 6 platos — S/ 148.60 — 497 Franklin Road

- 4 × Lomo Saltado (S/ 25.90 c/u)
- 2 × Aji de Gallina (S/ 22.50 c/u)

#19860 — 5 platos — S/ 115.90 — 7242 E North Street

- 1 × Lomo Saltado (S/ 25.90 c/u)
- 4 × Aji de Gallina (S/ 22.50 c/u)

#19855 — 4 platos — S/ 103.60 — 820 West Street

Admin: pedidos por estado con sus platos y cantidades

Delivery Cloud

Menú Analítica ADMIN Salir

Analítica

Top restaurantes

RESTAURANTE	VENTAS	PEDIDOS	CALIFICACIÓN
-------------	--------	---------	--------------

Métricas de usuarios

USUARIO	PEDIDOS	GASTO PROMEDIO	ANTIGÜEDAD
Cliente Demo	2	S/ 122.70	0-30 días
Repartidor Demo	0	S/ 0.00	0-30 días
Admin Demo	0	S/ 0.00	0-30 días
Lic. Laura Valverde	2	S/ 128.80	0-30 días
Mauro Yuridia Orellana	2	S/ 37.15	0-30 días
Mauro Prieto Olivas	0	S/ 0.00	0-30 días
Pascual Ramón Puente Anguiano	1	S/ 128.00	0-30 días
Ana Luisa Rocha Bermúdez	3	S/ 97.67	0-30 días
Gustavo Cristal Arroyo	2	S/ 77.35	0-30 días
Teodoro Mota Romero	1	S/ 122.70	0-30 días

Anterior Página 1 de 2001 · 20,007 en total Siguiente

Admin: analitica con datos de Athena

Delivery Cloud

DELIVERY Salir

Panel de repartidor

Disponibles para jalar (6,630)

#6 — Sabor Criollo — 4829 Marlen Light — S/ 48.40 Jalar

#2 — Pizza Bella — 757 Powlowski Ville — S/ 64.00 Jalar

#23 — Sabor Criollo — 289 Yost Station — S/ 145.20 Jalar

#27 — Pizza Bella — 84523 8th Street — S/ 64.00 Jalar

#29 — Sabor Criollo — 6196 Haylee Crest — S/ 148.60 Jalar

#31 — Sabor Criollo — 374 Block Station — S/ 100.20 Jalar

#32 — Pizza Bella — 6335 Crist Lodge — S/ 96.00 Jalar

#34 — Pizza Bella — 8517 Jonas Loaf — S/ 128.00 Jalar

#37 — Sabor Criollo — 249 Sycamore Street — S/ 167.70 Jalar

#39 — Sabor Criollo — 27926 Klein Glen — S/ 48.40 Jalar

Delivery: pedidos disponibles y entregas en curso

Llamadas REST por microservicio

Servicio	Llamadas en el frontend	Ejemplos
MS1	5	login, registro, perfil, direcciones
MS2	9	listar restaurantes, ver menu, CRUD de platos, listar y crear resenas
MS3	8	crear pedido, listar pedidos, cambiar estado, tomar y entregar
MS4	4	tracking del pedido, dashboards de customer, delivery y admin
MS5	2	top de restaurantes, metricas de usuarios (consulta a Athena)

6. Despliegue en AWS

6.1 Recursos

Recurso	Detalle
PP-App-Tier	EC2 t2.micro, us-east-1a; NGINX + MS1 hasta MS5 con docker compose; Elastic IP
PP-App-Tier-2	EC2 t2.micro, us-east-1b; identica a la anterior
PP-DB-Tier	EC2 t2.micro, us-east-1a; MySQL, PostgreSQL y MongoDB en contenedores con volumenes persistentes
PP-Ingest-VM	EC2 t2.micro, us-east-1a; 3 contenedores de ingesta en Python
pp-alb-interno	Application Load Balancer interno, target group pp-app-tg (las dos VMs de App), health check GET /health cada 15s
VPC Link + API Gateway	HTTP API con integracion privada (VPC Link) hacia el listener del ALB; protocolo HTTPS hacia el cliente
AWS Amplify	Conectado al repositorio de GitHub (rama main): cada push compila y despliega la SPA con amplify.yml (Node 22)
S3, Glue y Athena	Bucket pp-ubereats-datalake-..., base ubereats_datalake y crawler pp-crawler, workgroup pp-workgroup

6.2 Red y seguridad

Security group	Entrada permitida
pp-alb-sg (ALB)	TCP 80 solo desde pp-vpclink-sg
pp-app-tier-sg (VMs de App)	TCP 80 desde el ALB (tambien desde internet, ver seccion 11); SSH restringido a una IP de administracion
pp-db-tier-sg (VM de BD)	TCP 3306, 5432 y 27017 solo desde los SG de App e Ingesta; sin ningun puerto abierto a internet
pp-ingest-sg (Ingesta)	Ninguna entrada

- La administracion de las VMs se hace con AWS Systems Manager (SSH sobre SSM), sin exponer el puerto 22 de la VM de BD.
- Las instancias usan el rol IAM del laboratorio (LabInstanceProfile): no hay claves de AWS en el codigo ni en los contenedores.
- El acceso a las bases de datos se comprobo desde internet: 22, 3306, 5432 y 27017 no responden.

6.3 Balanceo de carga

El ALB interno reparte las peticiones entre las dos VMs de App. Lo comprobamos con la cabecera *X-Served-By*: en 30 peticiones consecutivas al mismo endpoint, 14 las atendio app-1 y 13 app-2. Para verificar la tolerancia a fallos detuvimos NGINX en la segunda VM: el ALB la marco como unhealthy en unos 30

segundos y las 30 peticiones siguientes las atendio app-1, todas con codigo 200. Al reiniciar NGINX, la VM volvio a rotacion.

6.4 Como se actualiza

El codigo se despliega con git pull y docker compose up --build en cada VM (en segundo plano, con swap de 2.5 GB porque las VMs t2.micro tienen 1 GB de RAM), y despues se reinicia NGINX, que resuelve las IPs de los contenedores al arrancar. El frontend se despliega solo: cada push a la rama main de GitHub dispara en Amplify la compilacion y el despliegue.

6.5 Documentacion Swagger

Servicio	URL de Swagger UI (a traves del API Gateway)
MS1	https://jt3z0elcud.execute-api.us-east-1.amazonaws.com/ms1/docs
MS2	https://jt3z0elcud.execute-api.us-east-1.amazonaws.com/ms2/swagger-ui.html
MS3	https://jt3z0elcud.execute-api.us-east-1.amazonaws.com/ms3/api-docs/
MS4	https://jt3z0elcud.execute-api.us-east-1.amazonaws.com/ms4/docs
MS5	https://jt3z0elcud.execute-api.us-east-1.amazonaws.com/ms5/docs

Detras de NGINX y del API Gateway los servicios FastAPI viven bajo un prefijo (/ms1, /ms4, /ms5). NGINX envia la cabecera X-Forwarded-Prefix y un middleware la usa como root_path, de modo que Swagger UI carga el esquema y el boton Try it out arma bien las URLs.

7. Data Science: ingesta, catalogo y analitica

7.1 Ingesta y data lake

La MV de ingesta (PP-Ingest-VM) ejecuta **tres contenedores Docker en Python**, uno por microservicio, con estrategia *pull* del 100% de las tablas. Cada uno genera archivos CSV (MySQL y PostgreSQL) o JSONL (MongoDB) y los sube al bucket de S3 en una carpeta por tabla y fecha (raw/mysql/usuarios/dt=AAAA-MM-DD/...). Antes de subir, cada contenedor elimina el snapshot anterior de esa tabla (full refresh), para que Athena no sume dos particiones.

Contenedor	Origen	Destino en S3	Formato
ingest-mysql	usuarios, direcciones (MS1)	raw/mysql/usuarios/ y raw/mysql/direcciones/	CSV
ingest-postgres	orders, order_items (MS3)	raw/postgres/orders/ y raw/postgres/order_items/	CSV
ingest-mongodb	restaurantes (MS2)	raw/mongodb/restaurantes/	JSONL

7.2 Catalogo de datos en AWS Glue

Un crawler (pp-crawler) recorre el bucket y crea una tabla por cada carpeta: **usuarios, direcciones, orders, order_items y mongodb** (el crawler nombro así a la colección de restaurantes). Los archivos no tienen claves foráneas; las relaciones del diagrama son lógicas y se resuelven con JOIN en Athena.

Diagrama Entidad-Relación del Catálogo de Datos — AWS Glue · base ubereats_datalake

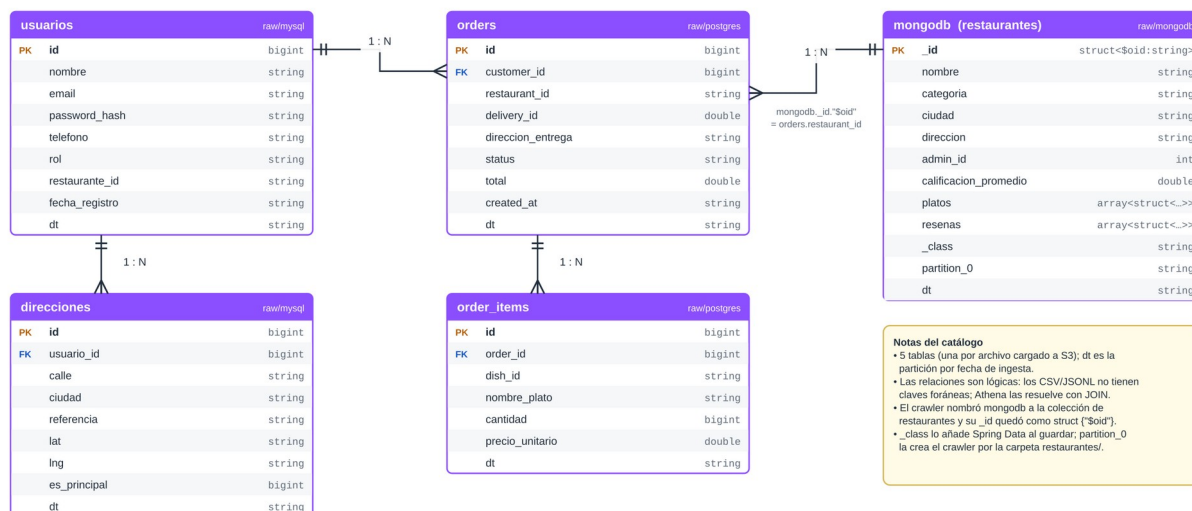


Figura 5. Diagrama E/R de las tablas del catalogo de Glue.

Hallazgos al catalogar: (1) el _id de MongoDB llegó como struct {"\$oid"} y se accede con r._id."\$oid"; (2) es_principal (booleano en MySQL) llegó como bigint 0/1; (3) Spring Data agrega el campo _class a los documentos de MongoDB y el crawler agrega la particion partition_0; (4) el modelo de usuarios no tiene fecha de nacimiento, por lo que la vista de metricas usa la antigüedad de la cuenta en lugar de una edad inventada.

7.3 Consultas en Amazon Athena

Las consultas se ejecutaron en el workgroup pp-workgroup sobre la base ubereats_datalake. Cada resultado indica el ID de ejecucion de Athena, que puede verificarse en el historial de consultas de la consola. Conteos del catalogo: usuarios = 20,007, orders = 20,009, order_items = 26,566, mongodb = 20,000.

Consulta 1 — Ventas por restaurante

Une orders, order_items y mongodb (3 tablas) para calcular pedidos y ventas totales por restaurante.

```
SELECT
  r.nombre AS restaurante,
  COUNT(DISTINCT o.id) AS num_pedidos,
  SUM(o.i.cantidad * oi.precio_unitario) AS total_ventas
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
JOIN mongodb r ON r._id."$oid" = o.restaurant_id
GROUP BY r.nombre
ORDER BY total_ventas DESC;
```

restaurante	num_pedidos	total_ventas
Sabor Criollo	9,898	997,825.70
Pizza Bella	10,111	806,688.00

Athena: 6568912e-2abe-4bf4-8e09-ba40eaf7c46c · 2.5s · 25.8 MB escaneados · primeras 6 filas de 2 devueltas.

The screenshot shows the Amazon Athena console interface. On the left, the 'Datos' sidebar lists the data source 'AwsDataCatalog', the database 'ubereats_datalake', and a list of tables including 'direcciones', 'mongodb', 'order_items', 'orders', 'usuarios', 'v_mtricas_usuarios', and 'v_resumen_ventas_restaurante'. The main area displays the SQL query for 'Consulta 1'. Below the query, the execution status is 'Completado' with a time of 1.926 seconds and 25.26 MB of data analyzed. The results table shows two rows: Sabor Criollo with 9,898 orders and 997,825.6999999991 in total sales, and Pizza Bella with 10,111 orders and 806,688.0 in total sales.

#	restaurante	num_pedidos	total_ventas
1	Sabor Criollo	9898	997825.6999999991
2	Pizza Bella	10111	806688.0

Captura de la consola de Amazon Athena (workgroup pp-workgroup).

Consulta 2 — Clientes con mayor gasto

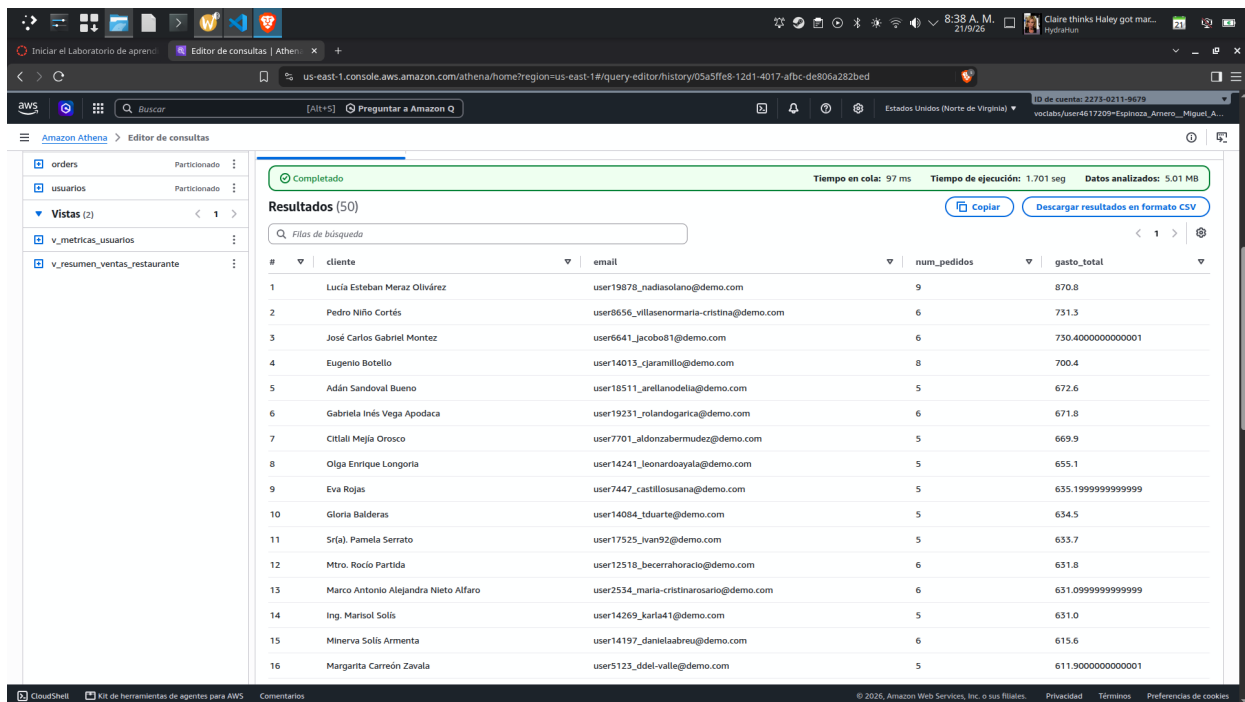
Une orders y usuarios para obtener el nombre real de los clientes que mas gastan.

```
SELECT
  u.nombre AS cliente,
  u.email,
  COUNT(o.id) AS num_pedidos,
```

```

SUM(o.total) AS gasto_total
FROM orders o
JOIN usuarios u ON u.id = o.customer_id
GROUP BY u.nombre, u.email
ORDER BY gasto_total DESC
LIMIT 50;

```



Completado

Tiempo en cola: 97 ms Tiempo de ejecución: 1.701 seg Datos analizados: 5.01 MB

Resultados (50)

Q Filas de búsqueda

#	cliente	email	num_pedidos	gasto_total
1	Lucia Esteban Meraz Olivarez	user19878_nadiasolano@demo.com	9	870.8
2	Pedro Niño Cortés	user8656_villasenormaria-cristina@demo.com	6	731.3
3	José Carlos Gabriel Montez	user6641_jacobo81@demo.com	6	730.4000000000001
4	Eugenio Botello	user14013_cjaramillo@demo.com	8	700.4
5	Adán Sandoval Bueno	user18511_arellanodelia@demo.com	5	672.6
6	Gabriela Inés Vega Apodaca	user19251_rolandogarica@demo.com	6	671.8
7	Citlali Mejía Orozco	user7701_aldonzabermudez@demo.com	5	669.9
8	Olga Enrique Longoria	user14241_leonardoayala@demo.com	5	655.1
9	Eva Rojas	user7447_castillosusana@demo.com	5	635.1999999999999
10	Gloria Balderas	user14084_tduarte@demo.com	5	634.5
11	Sr(a). Pamela Serrato	user17525_jvan92@demo.com	5	633.7
12	Mtro. Rocio Partida	user12518_becerrahoracio@demo.com	6	631.8
13	Marco Antonio Alejandra Nieto Alfaro	user2534_maria-cristinarosario@demo.com	6	631.0999999999999
14	Ing. Marisol Solís	user14269_karla41@demo.com	5	631.0
15	Minerva Solís Armenta	user14197_danielaabreu@demo.com	6	615.6
16	Margarita Carreón Zavala	user5123_didel-valle@demo.com	5	611.9000000000001

Captura de la consola de Amazon Athena (workgroup pp-workgroup).

Consulta 3 — Platos mas vendidos

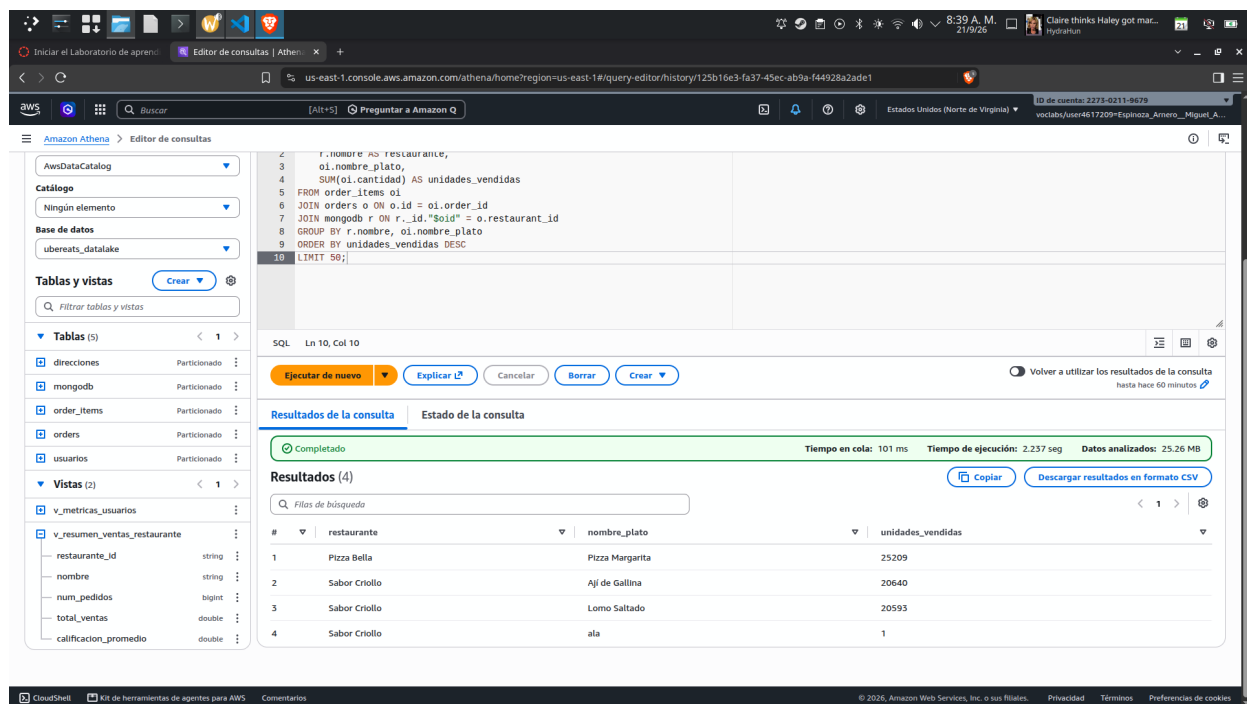
Une order_items, orders y mongodb (3 tablas) para obtener las unidades vendidas por plato y restaurante.

```

SELECT
    r.nombre AS restaurante,
    oi.nombre_plato,
    SUM(oi.cantidad) AS unidades_vendidas
FROM order_items oi
JOIN orders o ON o.id = oi.order_id
JOIN mongodb r ON r._id."$oid" = o.restaurant_id
GROUP BY r.nombre, oi.nombre_plato
ORDER BY unidades_vendidas DESC
LIMIT 50;

```

restaurante	nombre_plato	unidades_vendidas
Pizza Bella	Pizza Margarita	25,209
Sabor Criollo	Aji de Gallina	20,640
Sabor Criollo	Lomo Saltado	20,593



Captura de la consola de Amazon Athena (workgroup pp-workgroup).

Consulta 4 — Ciudad registrada vs. direccion de entrega

Une orders, usuarios y direcciones (LEFT JOIN) para comparar la ciudad del cliente con el destino del pedido.

```

SELECT
  u.nombre AS cliente,
  d.ciudad AS ciudad_registrada,
  o.direccion_entrega,
  COUNT(o.id) AS num_pedidos
FROM orders o
JOIN usuarios u ON u.id = o.customer_id
LEFT JOIN direcciones d ON d.usuario_id = u.id AND d.es_principal = 1
GROUP BY u.nombre, d.ciudad, o.direccion_entrega
ORDER BY num_pedidos DESC
LIMIT 50;
  
```

7.4 Vistas creadas

Se crearon dos vistas, consumidas por el microservicio MS5. Su resultado se muestra con un SELECT sobre cada vista.

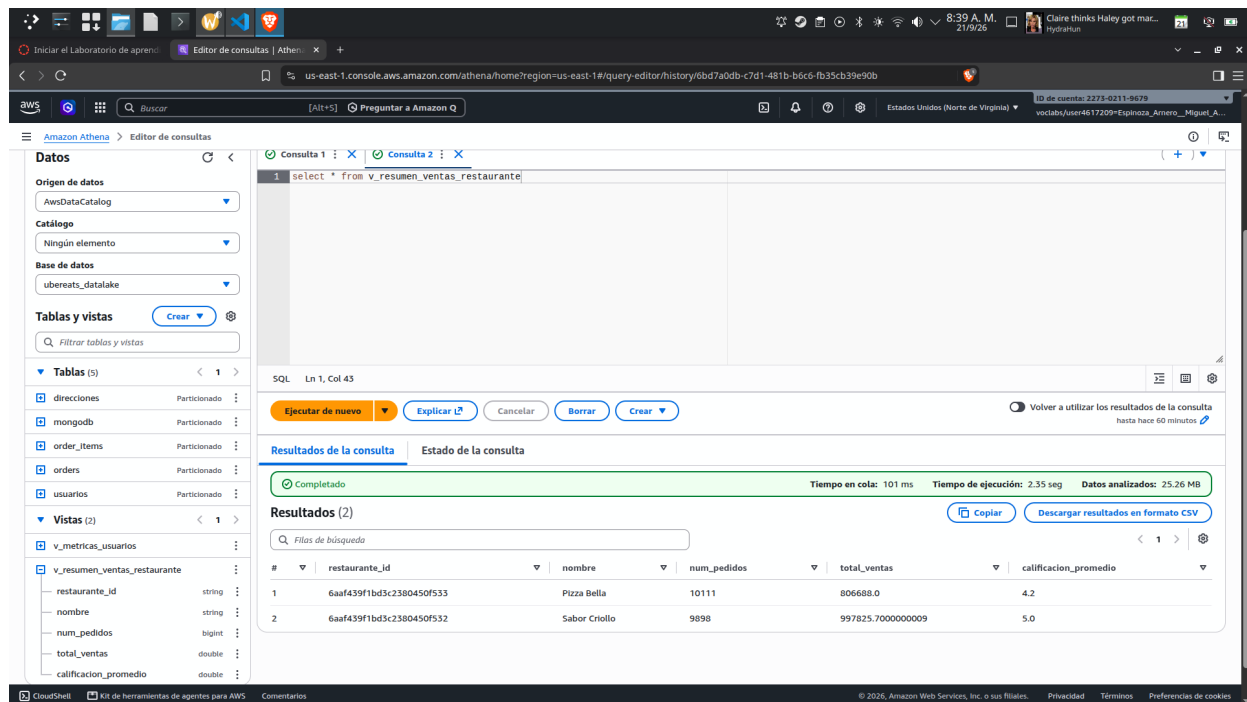
v_resumen_ventas_restaurante

```

CREATE OR REPLACE VIEW v_resumen_ventas_restaurante AS
SELECT
  r._id.$oid AS restaurante_id,
  r.nombre,
  COUNT(DISTINCT oi.id) AS num_pedidos,
  SUM(oi.cantidad * oi.precio_unitario) AS total_ventas,
  r.calificacion_promedio
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
JOIN mongodb r ON r._id.$oid = o.restaurant_id
GROUP BY r._id.$oid, r.nombre, r.calificacion_promedio;
  
```

restaurante_id	nombre	num_pedidos	total_ventas	calificacion_pro medio
6aaf439f1bd3c2380450f532	Sabor Criollo	9,898	997,825.70	5.00
6aaf439f1bd3c2380450f533	Pizza Bella	10,111	806,688.00	4.20

CREATE VIEW: ac099df1-4664-43b3-98d6-750b933e0c1c · SELECT: 4aedf6fe-5bcc-43d7-86e5-106e51f5a3c8 · 2.2 s.



Captura de la consola de Amazon Athena (workgroup pp-workgroup).

v_metricas_usuarios

CREATE OR REPLACE VIEW v_metricas_usuarios AS
SELECT

```

    u.id AS usuario_id,
    u.nombre,
    COUNT(o.id) AS num_pedidos,
    COALESCE(AVG(o.total), 0) AS gasto_promedio,
    CASE
        WHEN date_diff('day', date(substr(u.fecha_registro, 1, 10)),
current_date) < 30 THEN '0-30 dias'
        WHEN date_diff('day', date(substr(u.fecha_registro, 1, 10)),
current_date) < 90 THEN '31-90 dias'
        ELSE '90+ dias'
    END AS antiguedad_cuenta
FROM usuarios u
LEFT JOIN orders o ON o.customer_id = u.id
GROUP BY u.id, u.nombre, u.fecha_registro;
```

usuario_id	nombre	num_pedidos	gasto_promedio	antiguedad_cuenta
1	Cliente Demo	2	122.70	0-30 dias

usuario_id	nombre	num_pedidos	gasto_promedio	antiguedad_cuenta
2	Repartidor Demo	0	0.00	0-30 días
3	Admin Demo	0	0.00	0-30 días
4	Lic. Laura Valverde	2	128.80	0-30 días
5	Mauro Yuridia Orellana	2	37.15	0-30 días
6	Mauro Prieto Olivas	0	0.00	0-30 días

CREATE VIEW: 13e4a748-b0ae-477a-ab76-4d5faa49c729 · SELECT: 8f42b3a6-307a-4c46-8e36-f795d3f5293a · 2.0 s.

8. Paginado y rendimiento

Con mas de 20,000 registros por base, devolver listados completos era inviable (cientos de KB o MB por peticion y miles de elementos en el DOM). Anadimos un paginado opcional en la misma ruta, que no rompa a los consumidores existentes:

- Sin page ni page_size la respuesta es la de siempre (un arreglo con todo).
- Con ellos la respuesta es {items, page, page_size, total, total_pages}. page_size vale 20 por defecto y se limita a 100; valores invalidos dan 400 (Express y Spring) o 422 (FastAPI).
- El total tambien viaja en la cabecera X-Total-Count (expuesta por CORS). Los dashboards de MS4 y la analitica de MS5 tienen su propio modo paginado; MS3 agrega /orders/summary para contar pedidos por estado sin traer las filas.

Peticion	Sin paginar	Paginada
Dashboard admin (MS4)	4.2 MB, 0.84 s	4 KB, 60 ms
Usuarios (MS1)	6.6 MB, 2.3 s	Pagina de 20, 14 ms
Restaurantes (MS2, 20,000)	21 MB, 1.0 s	Pagina de 12, 12 KB, 14 ms
Pedidos de un restaurante (MS3)	Miles de filas	Pagina de 10, 10 ms
Metricas de usuarios (MS5)	Consulta completa a Athena	Pagina de 10, alrededor de 3 a 4s (latencia de Athena)

Mediciones tomadas en el entorno local con unos 40 mil pedidos (MS2 con sus 20,000 restaurantes); en AWS las respuestas son de un orden similar.

9. Pruebas y validacion

Prueba	Alcance	Resultado
Coleccion Postman + Newman	73 requests sobre los 5 servicios: flujo completo, casos de error (400, 401, 422) y modos con y sin paginado	73 ejecutadas, 69 aserciones, 0 fallos, en local y a traves del API Gateway
Navegador (Playwright), recorrido E2E	Registro de customer, pedido, resena, despacho del admin, toma y entrega del repartidor, tracking ENTREGADO, analitica y rutas protegidas, sobre la URL de Amplify y el backend en AWS (pruebas/e2e_full.mjs)	23 de 23 comprobaciones correctas; el unico error de consola (409) es a proposito, al intentar tomar dos veces el mismo pedido
Balanceo y failover	30 peticiones repartidas; NGINX detenido en una VM	14 de 13 entre las dos VMs; con una caida, 30 de 30 respuestas 200
Aislamiento de la BD	Sondeo de puertos 22, 3306, 5432 y 27017 desde internet	Todos cerrados

10. Seguridad

- Autenticacion con JWT (HS256, secreto de al menos 32 bytes) y contraseñas con bcrypt; autorizacion por rol en cada endpoint sensible.
- Red: BD sin puertos abiertos a internet, ALB interno, ingesta sin entradas y administracion sin SSH publico.
- Sin secretos en el repositorio: los archivos .env estan en .gitignore y en AWS se usa el rol de la instancia.
- Observacion: como la ingesta extrae el 100% de las tablas, password_hash llega al data lake. En produccion se excluiria o enmascararia esa columna antes de subirla.

11. Limitaciones y trabajo futuro

- **Puerto 80 de las VMs de App.** El trafico de produccion entra por API Gateway → ALB, pero las VMs aun aceptan el puerto 80 desde internet (por ejemplo por la IP elastica). Cerrarlo es una regla de red.
- **Recursos limitados.** Las VMs t2.micro (1 GB de RAM) funcionan con swap; en produccion convendria t3.small o superior.
- **Resenas.** MS2 toma el usuario_id del cuerpo de la peticion y no del token; deberia derivarse del JWT.
- **Data lake.** La ingesta es manual (bajo demanda); se podria programar con EventBridge y automatizar el crawler.

12. Lecciones aprendidas

- En AWS Academy Learner Lab las credenciales y las sesiones vencen, y al terminar la sesion las instancias quedan detenidas; los servicios se disenaron para reiniciar solos (restart: unless-stopped).
- La plataforma limita a 9 el numero de instancias de la cuenta y cuenta tambien las detenidas: las instancias nuevas se terminan a los segundos si se supera el limite.
- La AMI de Cloud9 trae Apache (puerto 80) y MySQL (3306) instalados: hay que deshabilitarlos antes de levantar NGINX y la base de datos.
- NGINX resuelve las IPs de los contenedores al arrancar: tras recrear un contenedor hay que reiniciarlo.
- Un crawler de Glue nombra las tablas segun la estructura de S3 y puede convertir tipos (booleano a bigint, ObjectId a struct): conviene revisar el catalogo antes de escribir consultas.

Anexo A. Enlaces

Elemento	Enlace
Repositorio publico (codigo fuente de los 5 microservicios, frontend, ingesta y diagramas)	https://github.com/miguelespinozaa-ship-it/ProeyctParcialProp
Aplicacion web (AWS Amplify)	https://main.d2wzxjgeu63fi4.amplifyapp.com
API publica (API Gateway, HTTPS)	https://jt3z0elcud.execute-api.us-east-1.amazonaws.com
Coleccion Postman	postman/delivery-cloud.postman_collection.json (en el repositorio)
Diagramas editables (draw.io)	infra/diagramas/proyecto-delivery-cloud.drawio (en el repositorio)

Anexo B. Como reproducir el proyecto

En local (cualquier persona que clone el repositorio):

```
git clone https://github.com/miguelespinozaa-ship-it/ProeyctParcialProp.git
cd ProeyctParcialProp
cp .env.example .env
docker compose -f docker-compose.dev.yml up -d --build
# servicios en http://localhost/ms1 ... /ms5
# Swagger: /ms1/docs /ms2/swagger-ui.html /ms3/api-docs /ms4/docs /ms5/docs
# frontend: cd frontend && npm install && npm run dev
# pruebas: npx newman run postman/delivery-cloud.postman_collection.json --env-var "base_url=http://localhost"
```

En AWS: las VMs de App usan docker-compose.app.yml, la VM de BD docker-compose.db.yml y la de ingesta data-science/docker-compose.ingest.yml. Los pasos de red (security groups, ALB, VPC Link y API Gateway) y los comandos de despliegue estan descritos en el README del repositorio.